Development Workflow for RBAC Feature

Project: Role-Based Access Control (RBAC) Implementation

Objective: Plan and structure the development of an enterprise RBAC feature allowing Admins to assign roles (Admin, Manager, Viewer) with distinct permissions.

1. Use Cases

Selected based on clarity and coverage of the core authorization flow.

Use Case 1: Admin Assigns Role

- **Actor:** System Administrator
- **Action:** The Admin navigates to the User Management dashboard, selects a user with "Viewer" access, and changes their role to "Manager" via the dropdown menu.
- **Outcome:** The system updates the user's record in the database, invalidates the user's current session (or updates the token), and the user gains access to Manager-level features immediately or upon next login.

Use Case 2: Manager Accesses Protected Resource

- **Actor:** Manager
- **Action:** A user with the "Manager" role attempts to access the "Quarterly Reports" page, which is restricted to Managers and Admins.
- **Outcome:** The backend middleware verifies the JWT token contains the "Manager" role claim. The server returns a 200 OK status with the report data, and the Frontend renders the page.

Use Case 3: Viewer Denied Access

- **Actor:** Viewer
- **Action:** A user with the "Viewer" role attempts to access the "System Settings" configuration page via a direct URL or bookmarked link.
- **Outcome:** The backend middleware detects insufficient privileges. The server returns a 403 Forbidden status. The Frontend catches this error and redirects the user to an "Unauthorized Access" error page explaining they lack permissions.

2. User Stories

Selected stories covering Admin, Manager, and Developer perspectives.

1. **Admin Perspective:**
   "As an **Admin**, I want to **assign specific roles (Admin, Manager, Viewer) to users**, so that **I can ensure employees only have access to the data necessary for their job functions.**"
2. **Manager Perspective:**
   "As a **Manager**, I want to **view the performance analytics dashboard**, so that **I can track my team's metrics without seeing system-wide configuration settings.**"
3. **Developer Perspective:**
   "As a **Developer**, I want to **implement a reusable RequireRole middleware**, so that **I can easily secure future API endpoints by simply adding a decorator or function call rather than rewriting logic.**"

3. Task Breakdown

A comprehensive list of 20 tasks categorized by domain, prioritized for development.

Task	Category	Priority	Estimate
1. Design Database Schema (Users, Roles, Permissions tables)	Backend	P0 (Critical)	4 hrs
2. Create Migration Scripts for initial Roles (Admin, Manager, Viewer)	Backend	P0 (Critical)	2 hrs
3. Implement POST /api/roles/assign Endpoint	Backend	P0 (Critical)	6 hrs
4. Implement JWT Token Update Logic (Add role claim to payload)	Backend	P0 (Critical)	4 hrs

5. Develop authMiddleware to verify token validity	Backend	P0 (Critical)	3 hrs
6. Develop roleMiddleware to check specific permissions	Backend	P1 (High)	5 hrs
7. Implement GET /api/users with Role data included	Backend	P1 (High)	3 hrs
8. Set up Frontend State Store (Redux/Context) to hold User Role	Frontend	P1 (High)	4 hrs
9. Create "User Management" UI (Table view for Admins)	Frontend	P1 (High)	8 hrs
10. Implement "Change Role" Dropdown Component	Frontend	P1 (High)	3 hrs
11. Create Protected Route Component (Higher-Order Component)	Frontend	P0 (Critical)	5 hrs
12. Design and Implement 403 (Unauthorized) Page	Frontend	P2 (Medium)	3 hrs
13. UI Conditional Rendering (Hide "Settings" button	Frontend	P2 (Medium)	4 hrs

for non-Admins)			
14. Unit Tests for roleMiddleware logic	Testing	P1 (High)	4 hrs
15. Integration Tests for Role Assignment Flow	Testing	P1 (High)	6 hrs
16. Secure Headers Implementation (Helmet, CORS config)	Security	P0 (Critical)	2 hrs
17. Input Validation (Sanitize Role inputs to prevent injection)	Security	P1 (High)	3 hrs
18. OWASP ZAP Scan (Automated vulnerability scanning)	Security	P2 (Medium)	4 hrs
19. Rate Limiting on Admin Endpoints (Prevent brute force)	Security	P2 (Medium)	3 hrs
20. Documentation (API Swagger update & Admin Guide)	Backend	P3 (Low)	4 hrs

4. Development Workflow

The following workflow outlines the lifecycle of the RBAC feature from planning to deployment, highlighting team roles and approval gates.

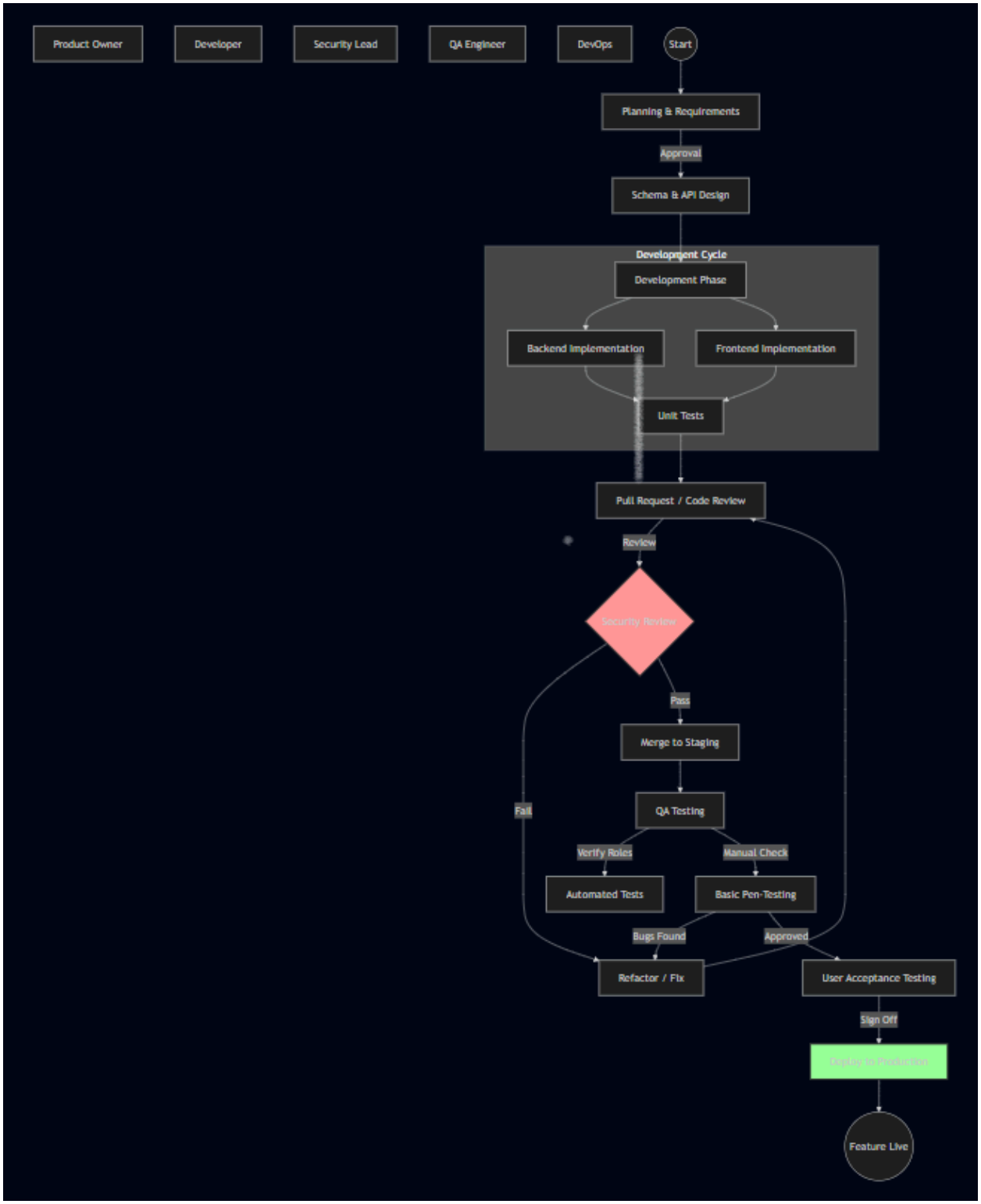
Workflow Stages

1. **Planning:** Product Owner defines roles; Tech Lead approves schema.
2. **Development:** Backend and Frontend developers work in parallel feature branches.
3. **Code Review:** Peers review code for logic errors; Security Lead reviews auth logic.
4. **Testing:** QA runs automated regression tests and manual role verification.
5. **Deployment:** CI/CD pipeline deploys to Staging for final acceptance, then Production.

Mermaid Diagram

```
graph TD
    %% Roles
    PO[Product Owner]
    Dev[Developer]
    Sec[Security Lead]
    QA[QA Engineer]
    Ops[DevOps]

    %% Flow
    Start((Start)) --> Plan[Planning & Requirements]
    Plan --> |Approval| Des[Schema & API Design]
    Des --> DevPhase[Development Phase]

    subgraph Development Cycle
        DevPhase --> BE[Backend Implementation]
        DevPhase --> FE[Frontend Implementation]
        BE --> Unit[Unit Tests]
        FE --> Unit
    end

    Unit --> PR[Pull Request / Code Review]
    PR --> |Review| SecCheck{Security Review}

    SecCheck -- Fail --> Fix[Refactor / Fix]
    Fix --> PR
    SecCheck -- Pass --> Merge[Merge to Staging]

    Merge --> TestPhase[QA Testing]
```

```
TestPhase --> |Verify Roles| AutoTest[Automated Tests]
TestPhase --> |Manual Check| PenTest[Basic Pen-Testing]

PenTest -- Bugs Found --> Fix
PenTest -- Approved --> UAT[User Acceptance Testing]

UAT --> |Sign Off| Deploy[Deploy to Production]
Deploy --> End((Feature Live))

%% Styling
style SecCheck fill:#ff9999,stroke:#333,stroke-width:2px
style Deploy fill:#99ff99,stroke:#333,stroke-width:2px
```


Product Owner
Developer
Security Lead
QA Engineer
DevOps
Start
Planning & Requirements
Approval
Schema & API Design
Development Cycle
Development Phase
Backend Implementation
Frontend Implementation
Unit Tests
Pull Request / Code Review
Review
Security Review
Pass
Merge to Staging
QA Testing
Fail
Verify Roles
Manual Check
Automated Tests
Basic Pen-Testing
Bugs Found
Approved
Refactor / Fix
User Acceptance Testing
Sign Off
Deploy to Production
Feature Live

5. Validation & Refinement

Refinements based on analysis of scalability (10k users), security (OWASP), and maintainability.

1. Scalability (10k+ Users)

- **Issue:** Checking roles against the database on every single API request will create a bottleneck as traffic grows to 10,000 concurrent users.
- **Refinement:** Implement **Redis Caching** for user sessions/roles. When a request comes in, check the cache first.
    - *Added to implementation:* When an Admin changes a role (Task 3), the system must now also **invalidate the Redis cache key** for that specific user to force a refresh.

2. Security (OWASP Compliance)

- **Issue:** Simply checking the role is insufficient if IDOR (Insecure Direct Object Reference) is ignored. A "Manager" might access *another* Manager's report by changing an ID in the URL.
- **Refinement:** Enhanced roleMiddleware (Task 6) to include **Resource Ownership Checks**.
    - *Rule:* Permission is not just Role = Manager, but Role = Manager AND (Resource.OwnerID == UserID OR Role == Admin).
- **Issue:** Broken Access Control is #1 on OWASP Top 10.
    - *Refinement:* Added **Task 15 (Integration Tests)** specifically to test *negative* cases (e.g., ensure Viewers explicitly *cannot* access Admin APIs).

3. Maintainability

- **Issue:** Hardcoding strings like "Admin" or "Manager" throughout the codebase makes it fragile and hard to update if role names change.
- **Refinement:** Create a **Centralized Constants/Enums File** (RoleConstants.js or RoleEnum.cs).
    - *Implementation:* All code references must use Roles.ADMIN instead of string literals.
    - *Documentation:* Added **Task 20** to ensure the "Matrix of Permissions" is documented for future developers onboarding to the project.